

EE69210: Machine Learning for Signal Processing Laboratory
Department of Electrical Engineering, Indian Institute of Technology, Kharagpur

Polynomial Regression

Anirvan Krishna | Roll no. 21EE38002

Keywords: Polynomial Regression, Normal Equations, Gradient Descent

Grading Rubric

	Tick the best applicable per row			Points
	Below Ex- pectations	Lacking in Some	Meets all Ex- pectations	
Completeness of the report				
Organization of the report (5 pts)				
Quality of figures (5 pts)				
Dataset preparation and explanation (10 pts)				
Feature scaling and train-test split and explanation (25 pts)				
Implementation of Gradient Descent algorithm and explanation (40 pts)				
Model Evaluation and Inference (15 pts)				
TOTAL (100 pts)				

1. Theory of Polynomial Regression

Polynomial regression is a type of regression analysis used to model the relationship between the independent variable x and the dependent variable y as an n -th degree polynomial function. Generally, x can be a multi-dimensional vector, but in this case, we consider x to be a scalar.

The polynomial regression model is expressed as follows:

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_n x^n \quad (1)$$

where: - $\hat{y}$ is the predicted value, - x is the input sample, - $\theta_0, \theta_1, \dots, \theta_n$ are the coefficients of the polynomial terms, - n is the degree of the polynomial.

In matrix form, the same equation can be rewritten as:

$$\hat{Y} = \mathbf{X}\Theta \quad (2)$$

where:

$$\mathbf{X} \in \mathbb{R}^{m \times (n+1)}, \quad \Theta \in \mathbb{R}^{(n+1) \times 1}, \quad \hat{Y} \in \mathbb{R}^{m \times 1}$$

with:

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^n \\ 1 & x_2 & x_2^2 & \dots & x_2^n \\ 1 & x_3 & x_3^2 & \dots & x_3^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m & x_m^2 & \dots & x_m^n \end{bmatrix}, \quad \Theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}, \quad \hat{Y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{bmatrix}, \quad Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

Here, m is the number of samples, and each row of $\mathbf{X}$ represents a unique sample with its polynomial features up to degree n . The vector Θ contains the model parameters, and $\hat{Y}$ holds the predicted values corresponding to each sample in $\mathbf{X}$.

2. Feature Scaling

To improve numerical stability and convergence during optimization, feature scaling is often applied. The standardization of each feature x is given by:

$$x' = \frac{x - \mu}{\sigma} \quad (3)$$

where: - x' is the scaled feature, - μ is the mean of the feature values, - σ is the standard deviation of the feature values.

Feature scaling ensures that all features contribute equally to the learning process, preventing dominance by features with larger magnitudes.

3. Solution Methods

Polynomial regression parameters Θ can be estimated using either *Gradient Descent* or the *Normal Equation*.

3.1 Gradient Descent Solution

Gradient descent is an iterative optimization algorithm used to minimize the cost function:

$$J(\Theta) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 \quad (1)$$

where m is the number of samples, $\hat{y}_i = \mathbf{X}_i\Theta$, and y_i is the true output.

The parameters are updated iteratively as follows:

$$\Theta := \Theta - \alpha \frac{1}{m} \mathbf{X}^T (\mathbf{X}\Theta - \mathbf{Y}) \quad (2)$$

where:

- α is the learning rate,
- $\mathbf{X}^T (\mathbf{X}\Theta - \mathbf{Y})$ is the gradient of $J(\Theta)$ with respect to Θ .

Gradient descent iterates until convergence, ensuring a minimal cost function value.

3.2 Normal Equation Solution

The Normal Equation provides a closed-form solution by directly computing the optimal Θ as:

$$\Theta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y} \quad (3)$$

This method does not require iterative updates but involves computing the inverse of $\mathbf{X}^T \mathbf{X}$, which may be computationally expensive for large datasets.

4. Dataset for Polynomial Regression

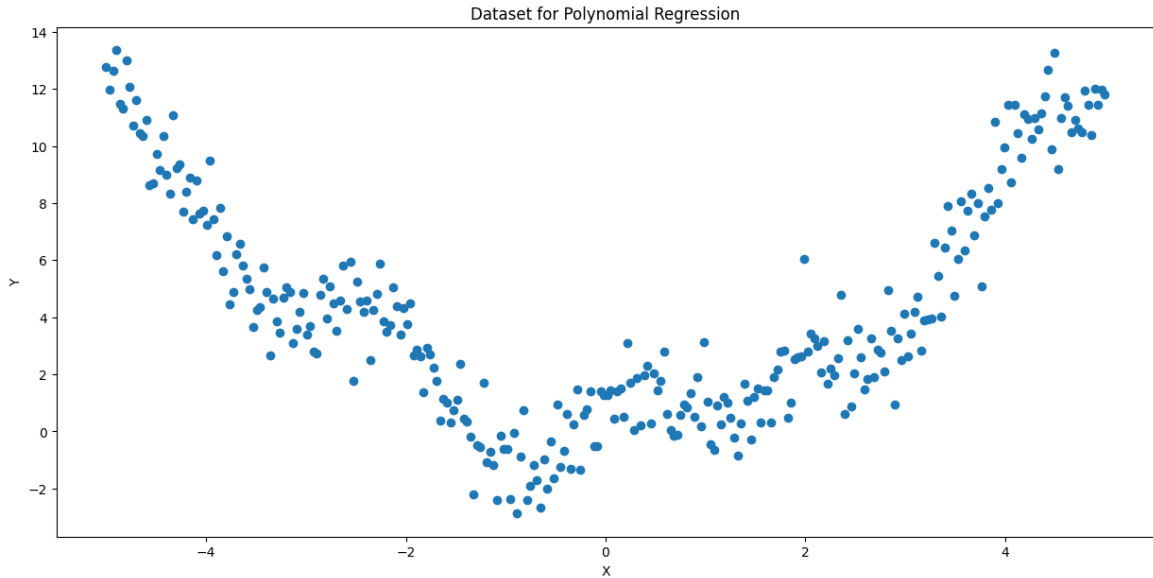


Fig. 1. Dataset for Polynomial Regression

5. Code for Polynomial Regression

```
1 class PolynomialRegression:
2
3     def __init__(self, degree):
4         self.degree = degree
5         self.coefficients = None
6         self.gd_loss = []
7         self.gd_mae = []
8         self.gd_mape = []
9
10    def fit_normal_equation(self, x, y):
11        X = np.vander(x.flatten(), N=self.degree + 1, increasing=True)
12        self.coefficients = np.linalg.inv(X.T.dot(X)).dot(X.T).dot(y)
13
14    def fit_gradient_descent(self, x, y, learning_rate=0.01, n_iters=1000):
15
16        X = np.vander(x.flatten(), N=self.degree + 1, increasing=True)
17        n_samples, n_features = X.shape
18        self.coefficients = np.random.randn(n_features, 1)
19
20        for _ in range(n_iters):
21
22            y_pred = X.dot(self.coefficients)
23            residuals = y_pred - y
24            loss = np.mean(residuals**2)
25            mae = mean_absolute_error(y, y_pred)
26            mape = mean_absolute_percentage_error(y, y_pred)
27            self.gd_loss.append(loss)
28            self.gd_mae.append(mae)
29            self.gd_mape.append(mape)
30
31            gradients = 1/n_samples * X.T.dot(X.dot(self.coefficients) - y)
32            self.coefficients -= learning_rate * gradients
33
34    def predict(self, x):
35        X = np.vander(x.flatten(), N=self.degree + 1, increasing=True)
36        return X.dot(self.coefficients)
```

6. Observations and Results

Method	MAE	MAPE	MSE
Normal Equation	1.07	0.80	1.82
Gradient Descent	1.08	1.16	1.65

Table 1. Evaluation metrics for Normal Equation and Gradient Descent solutions

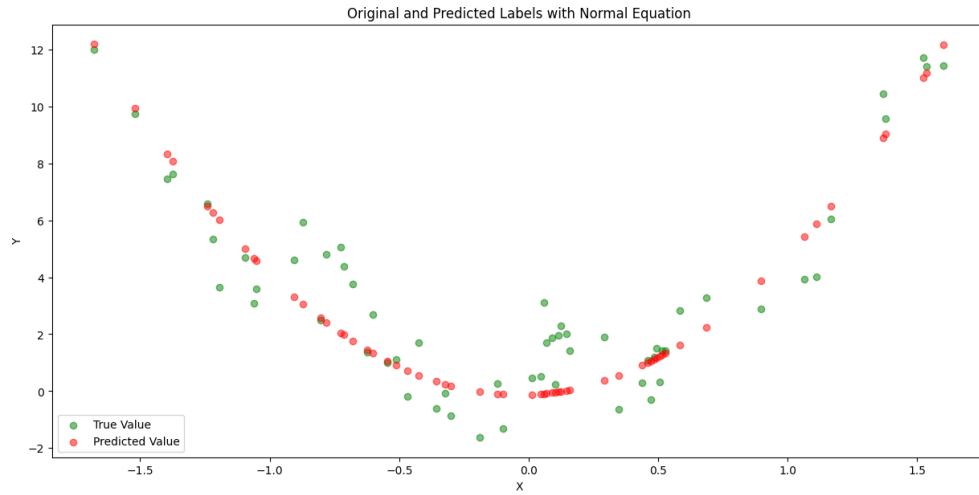


Fig. 2. Plots for test set predictions generated using normal equation solutions

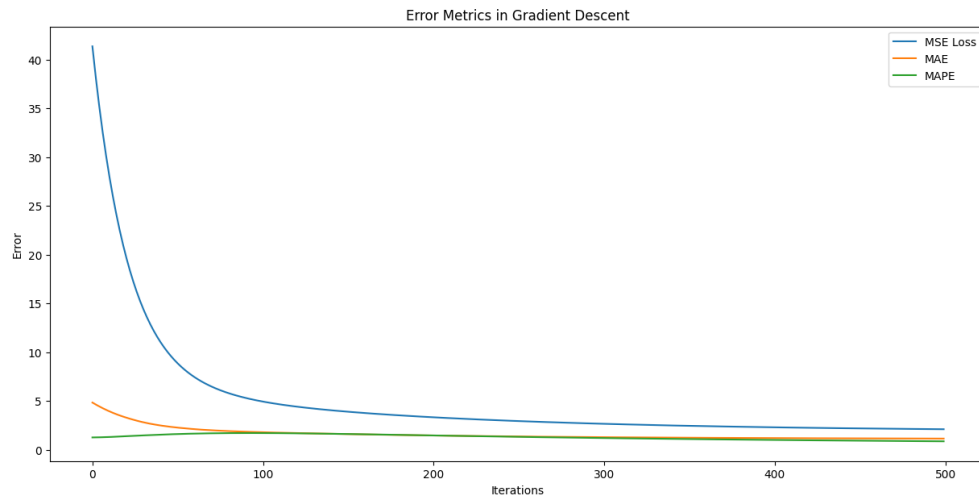


Fig. 3. Error trajectory for gradient descent solutions

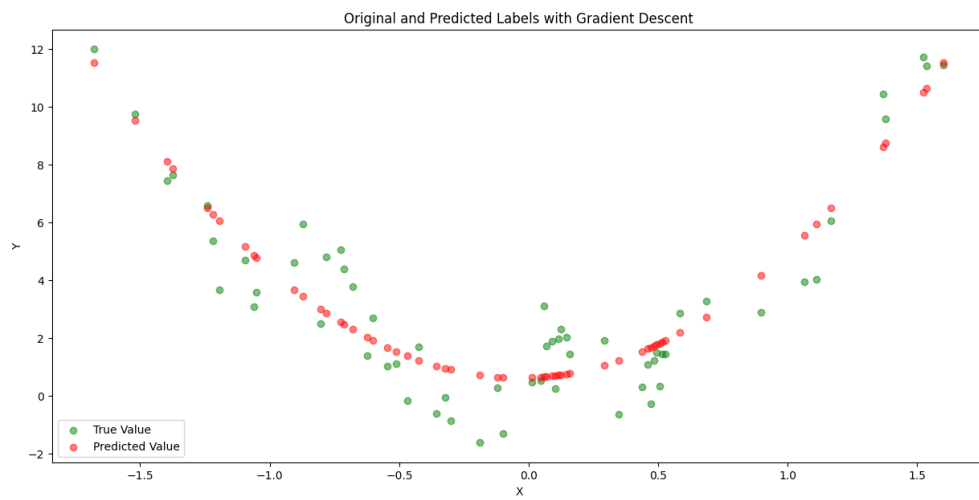


Fig. 4. Plots for test set predictions generated using gradient descent solutions